Introduction to Computer Systems

Lecture 11: Networking and Remote Procedure Calls

Multiplexing, Addressing, Reliability, Layering, and RPC Semantics

Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Contents

1 Introduction **3**

2 Why Networking Matters **3**
2.1 Networked Computing as the Common Case . . . 3
2.2 Basic Network Functionalities . . . 3

3 Links, Multiplexing, and Packet Switching **5**
3.1 Physical Links and Link-Layer Coordination . . . 5
3.2 Multiplexing as Network Time Sharing . . . 5
3.3 Circuit Switching versus Packet Switching . . . 6

4 Addressing Across Layers **8**
4.1 From Hosts to Interfaces to Processes . . . 8
4.2 Common Port Numbers . . . 8

5 Local Area Networks: Broadcast, ARP, Hubs, and Switches **9**
5.1 Broadcast and ARP . . . 9
5.2 Hubs, Collisions, and Shared Media . . . 11
5.3 Switches and Learning . . . 11

6 Wide Area Networks and Routers **11**
6.1 From LANs to WANs . . . 11
6.2 Packet Forwarding and Store-and-Forward . . . 12
6.3 Routing Policy and Business Constraints . . . 12

7 DNS: Names as Indirection **13**
7.1 Name Resolution . . . 13
7.2 Caching, TTL, and Staleness . . . 13
7.3 DNS as an Overloaded Mechanism . . . 15

8 Reliability, Flow Control, and Congestion Control **15**
8.1 Communication Channel Parameters . . . 15
8.2 Loss, Corruption, and Reordering . . . 16
8.3 Network Overload . . . 16
8.4 Stop-and-Wait and the Cost of RTT . . . 16
8.5 Sliding Windows and TCP Congestion Control . . . 19
8.6 When TCP Is Not the Right Abstraction . . . 20

9 Layering and the Internet Hourglass **22**
9.1 Why Layering Exists . . . 22
9.2 The Five-Layer Internet Model . . . 22
9.3 The Internet Hourglass . . . 23
9.4 Drawbacks of Layering . . . 23

10 The End-to-End Argument **25**
10.1 Where Should Functionality Live? . . . 25

11 Inter-Process Communication and Sockets **26**
11.1 IPC on One Machine and Across the Network . . . 26
11.2 The Socket API . . . 26
11.3 Client-Server and Peer-to-Peer Modes . . . 28

12 Remote Procedure Calls **28**
12.1 From Raw Bytes to Typed Calls . . . 28
12.2 Stubs, Marshalling, and Unmarshalling . . . 28
12.3 IDL and gRPC . . . 28

13 RPC Failure Semantics **31**
13.1 RPC Is Not a Local Call . . . 31
13.2 At Least Once, At Most Once, and Exactly Once . . . 32
13.3 Idempotence . . . 34

14 Summary **34**

15 Quizzes & Exercises **35**
15.1 Short-Answer Quiz . . . 35
15.2 Calculation Exercises . . . 36
15.3 Design Questions . . . 36
15.4 Programming Exercises . . . 36

1 Introduction

Computer networking is the part of computer systems that allows independent machines and independent processes to communicate. It is easy to underestimate networking because a modern user often sees only a browser, an app, a cloud drive, or a chat window. Beneath that apparent simplicity is a layered system of physical media, link technologies, addressing mechanisms, routing policies, transport protocols, naming services, congestion-control algorithms, and application-level protocols. The central achievement of the Internet is that these pieces interoperate despite being built by different vendors, administered by different organizations, and deployed over technologies ranging from copper wires and optical fibers to Wi-Fi, cellular networks, data-center fabrics, and satellite links.

This lecture develops the networking material needed for an operating-systems view of distributed software. The first half explains how the network moves packets: how links are shared, how addresses identify interfaces and processes, how local networks use broadcast and switches, how wide-area networks route packets, and how DNS maps names to addresses. The second half explains how software uses the network: what reliability and congestion control mean, why the layered architecture is powerful but imperfect, how sockets expose raw byte streams, and why remote procedure calls (RPCs) are a safer and more productive abstraction for many distributed programs.

Definition

A **protocol** is an agreement about communication. It specifies *syntax*, such as message formats, field order, encodings, and timers, and *semantics*, such as what each message means and what action a participant should take after sending, receiving, timing out, or detecting an error.

The key systems theme is resource sharing under uncertainty. Many hosts share the same physical media, many applications share the same network, many processes share the same host, and many organizations share the global routing system. Communication can be delayed, reordered, corrupted, duplicated, or lost. A robust network stack therefore does not merely send bytes; it builds structured abstractions on top of unreliable and heterogeneous mechanisms.

2 Why Networking Matters

2.1 Networked Computing as the Common Case

Many modern computing devices are valuable primarily because they connect to other machines. A phone without network service cannot synchronize calendars, stream video, use maps, call cloud APIs, fetch software updates, or interact with AI services. A laptop used for programming still depends on package repositories, source-code hosting, documentation sites, build servers, authentication services, and collaboration tools. A failure in networking often feels like a failure of the whole computer because many applications assume connectivity.

Networking is also a systems topic, not merely a communication-engineering topic. The same design principles that appear in operating systems reappear in networks: abstraction, modularity, scheduling, multiplexing, failure recovery, naming, caching, and concurrency. A network is a distributed resource manager. It must decide who may send, when they may send, how destinations are found, how failures are detected, and how overload is avoided.

2.2 Basic Network Functionalities

A practical network must provide several basic functionalities.



Basic Network Functionalities

- **Delivery**: deliver packets between to any two hosts in the Internet without a large number of physical wires
  - E.g., deliver a packet from a host in Beijing to a host in LA?
- **Reliability**: tolerate packet losses
  - E.g., how do you ensure all bits of a file are delivered in the presence of packet loses?
- **Flow control and congestion control**: avoid overflowing the receiver + avoid overflowing the routers
  - E.g., how do you ensure that a server that can send at 10Gbps doesn't overwhelm a LTE phone?
  - Flow control is for your data transfer
  - Congestion control is for the general good of the entire network

Figure 1: The basic functional goals of networking: delivery, reliability, flow control, and congestion control. Delivery asks whether packets can reach arbitrary hosts; reliability asks whether data survives loss and corruption; flow control protects the receiver; congestion control protects the network as a shared resource.

Delivery means that a packet can travel from one host to another without requiring a dedicated physical wire between every pair of hosts. If a host in Beijing sends a packet to a host in Los Angeles, intermediate switches and routers forward the packet hop by hop. This is the basic scaling problem: with n hosts, direct physical wiring between every pair would require $O(n^2)$ links, which is impossible at Internet scale.

Reliability means tolerating loss, corruption, reordering, and duplication. The physical network may drop packets, radio links may corrupt bits, routers may overflow, and paths may change. Protocols use checksums, sequence numbers, acknowledgments, retransmissions, and timers to build a more useful abstraction over this unreliable substrate.

Flow control prevents the sender from overwhelming the receiver. A data-center server may be able to transmit at tens or hundreds of gigabits per second, while a phone on a cellular link may receive much more slowly and have limited memory for buffering.

Congestion control prevents senders from overwhelming the network itself. If too many flows push traffic into a slow router or link, queues grow, packets are dropped, senders retransmit, and the resulting feedback loop can reduce useful throughput. Congestion control is therefore a public-good mechanism: each sender must restrain itself for the health of the shared network.

Deep Dive

Flow control and congestion control solve different overload problems. Flow control asks: "Can the receiver absorb this data?" Congestion control asks: "Can the path through the network absorb this data without harming everyone else?" A receiver can be fast while the network path is congested, and a network path can be uncongested while the receiver is slow.

3 Links, Multiplexing, and Packet Switching

3.1 Physical Links and Link-Layer Coordination

At the bottom of the network is a *link*: a physical or wireless communication medium connecting nodes. A link raises electrical and physical questions: what voltage level, optical intensity, frequency, or modulation represents a bit; how long a bit lasts; how the receiver synchronizes with the sender; and whether the medium is copper, fiber, radio, or something else.

Above the physical bit stream is the data-link problem. If multiple nodes share a medium, they need rules for when to transmit, how to identify the intended receiver, and how to detect or avoid simultaneous transmissions. Human conversation illustrates the issue: if two people begin speaking at once, their messages collide. On a short link, a transmitter may quickly detect that another node is transmitting. On a long link, propagation delay matters; a node may begin transmitting before hearing that another node has already started.

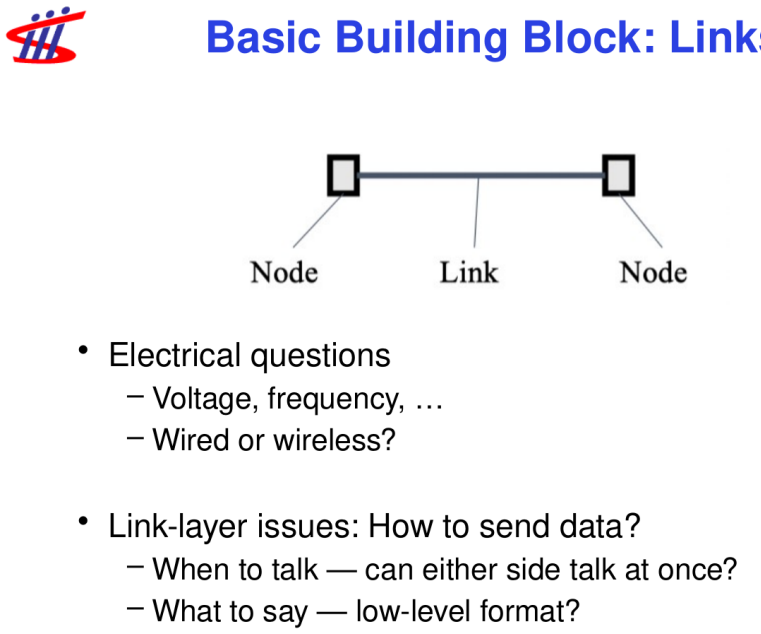


Figure 2: A link is the primitive communication building block. The physical layer is concerned with signals and media, while the link layer is concerned with sending frames, arbitrating access, and defining low-level formats.

3.2 Multiplexing as Network Time Sharing

Definition

Multiplexing is the sharing of one communication resource among multiple logical communications. It can occur in time, frequency, code space, wavelength, or statistical packet scheduling.

Multiplexing in networks is analogous to time sharing in operating systems. One CPU can run multiple processes by switching rapidly among them. One network link can carry many conversations by interleaving their data. If host A uses the link for a while and then host B uses it, both appear to have connectivity even though the physical resource is shared.

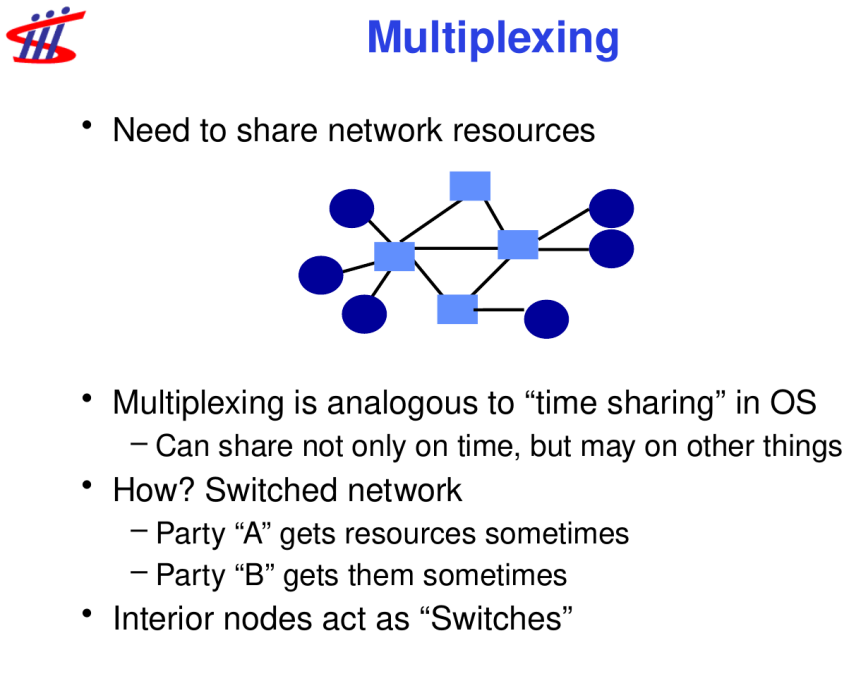


Figure 3: Multiplexing shares network resources among multiple parties. Instead of dedicating one wire to each communication pair, interior nodes switch traffic so that different parties use the resource at different times.

Time-division multiplexing assigns different time intervals to different flows. Frequency-division multiplexing assigns different frequency bands. Wavelength-division multiplexing assigns different optical wavelengths. Packet switching uses a more flexible form called *statistical multiplexing*: packets are sent when traffic exists, and idle users do not consume reserved capacity.

3.3 Circuit Switching versus Packet Switching

Early telephone networks were based on circuit switching. A call reserved a path through the network, and that path remained dedicated to the call until it ended. This provides predictable quality of service: once the circuit is established, the reserved resources are available. The cost is poor utilization when the parties are silent or sending little data.

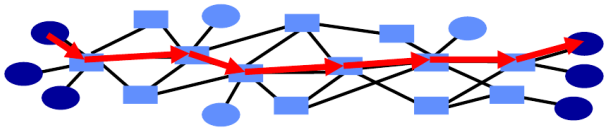
The Internet is based primarily on packet switching. A message is divided into packets, each packet contains an address, and each packet is forwarded independently. Packets from many flows are interleaved on shared links. This is why packet switching is efficient for bursty data traffic: a user consumes network capacity only when actually sending packets.

Packet switching trades guaranteed quality for higher average utilization. This tradeoff is fundamental. If the network is lightly loaded, packets move quickly. If it is overloaded, queues grow and packets are dropped. Higher-layer protocols must then detect and recover from loss, and senders must adjust rates to avoid collapse.



Packet Switching

- Source sends information as self-contained packets that have an address
  - Source may have to break up single message into multiple
- Each packet travels independently to the destination host
  - Switches use the address in the packet to detmerine how to forward the packets
  - Store and forward
- Analogy: a letter in surface mail.

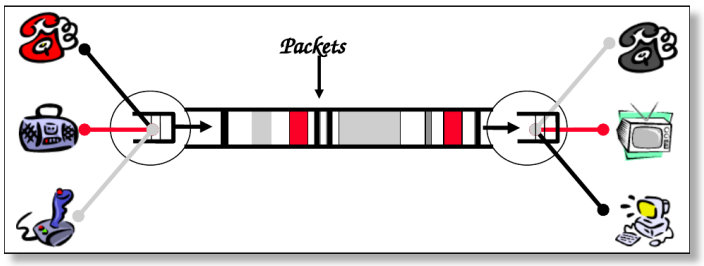


10 10

Figure 4: Packet switching sends self-contained packets with addresses. A large message may be divided into several packets; each packet is independently forwarded by switches and routers toward the destination.



Packet Switching — Statistical Multiplexing



- Swtiches arbitrate between inputs
- Can send from any input that's ready
  - Links never idle when there is traffic to send
  - Efficiency

11 11

Figure 5: Statistical multiplexing improves efficiency. Switches arbitrate among ready inputs; a link need not be idle merely because one logical conversation has nothing to send.

4 Addressing Across Layers

4.1 From Hosts to Interfaces to Processes

A network ultimately connects processes, not abstract machines. When a browser retrieves a web page, a browser process communicates with a web-server process. However, that process-to-process communication is built in stages. A packet first reaches a network interface card (NIC), then reaches a host, then reaches a process identified by a port number.

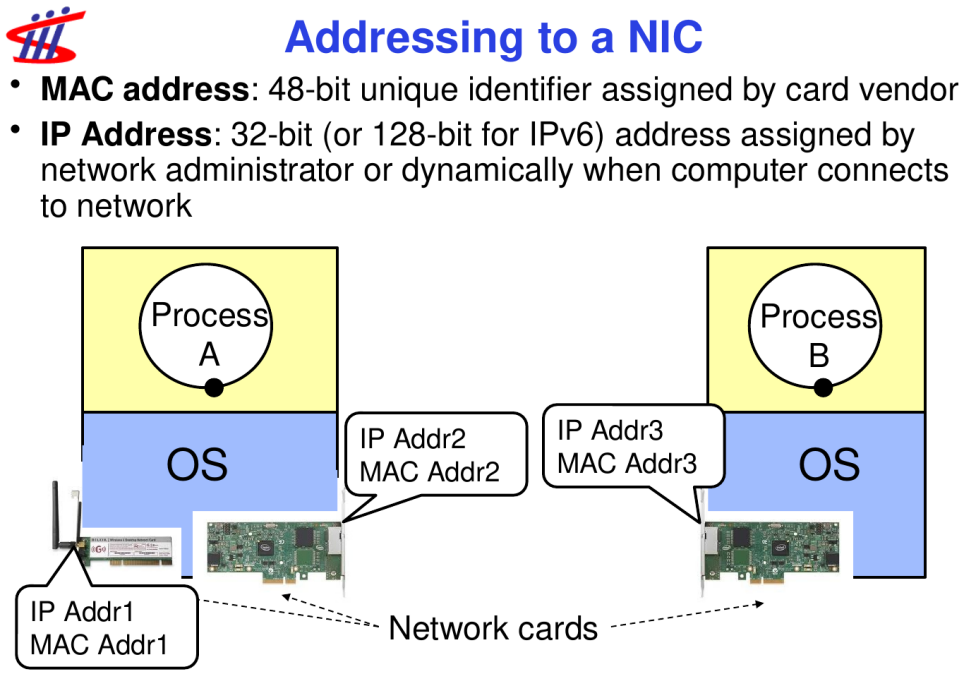


Figure 6: Addressing at the network-interface level. A NIC has a MAC address and may have one or more IP addresses assigned by administrators or dynamically when the device joins a network.

A **MAC address** is a data-link-layer identifier, historically associated with a network card. Ethernet MAC addresses are 48 bits. They are mostly flat identifiers: they do not describe where a host is located in the network topology. This makes MAC addresses suitable inside a local network, but unsuitable for global routing.

An **IP address** is a network-layer identifier. IPv4 addresses are 32 bits, and IPv6 addresses are 128 bits. Unlike MAC addresses, IP addresses are structured for routing. A router can inspect an address prefix and decide which direction a packet should go. A laptop has different IP addresses at school, at home, and on a phone hotspot because the IP address reflects network attachment.

A **port number** is a transport-layer identifier. It distinguishes processes or services on the same host. A TCP or UDP endpoint is identified by an IP address and port number. A connection is identified by the local and remote endpoint information, such as

$$(\text{source IP}, \text{source port}, \text{destination IP}, \text{destination port}).$$

4.2 Common Port Numbers

Well-known services conventionally use well-known ports. HTTP uses port 80, HTTPS uses port 443, SSH uses port 22, and DNS usually uses port 53. A server typically binds a fixed port and

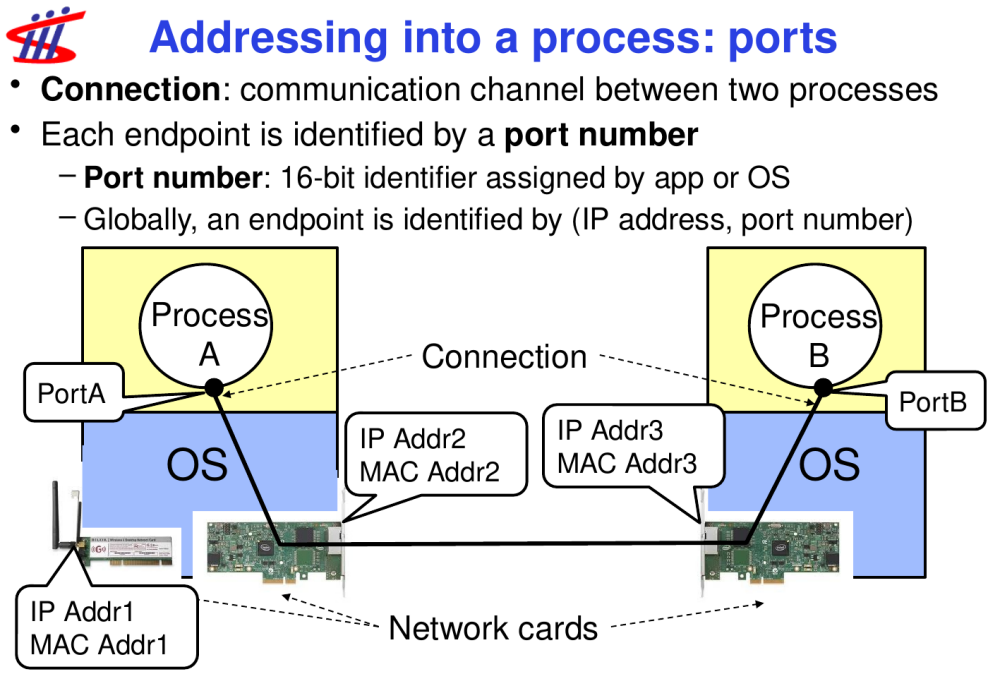


Figure 7: Addressing into a process. The network first reaches an interface through an IP address; the transport layer then demultiplexes traffic to a process using a port number.

listens for connections. A client usually receives an ephemeral port assigned by the operating system.

Table 1: Examples of well-known port numbers.

Application	Port number
Wake-on-LAN	9
FTP data	20
FTP control	21
SSH	22
Telnet	23
DNS	53
HTTP	80
HTTPS	443
SNMP	161

The layering of addresses avoids a single overloaded identifier. A MAC address finds a card in a local broadcast domain. An IP address finds an interface across networks. A port number finds a process on a host. Each address operates at a different scope.

5 Local Area Networks: Broadcast, ARP, Hubs, and Switches

5.1 Broadcast and ARP

Inside a local area network (LAN), hosts can often broadcast a frame that all local participants receive. Broadcast makes it possible to resolve an IP address into a MAC address using ARP,

the Address Resolution Protocol. If a host wants to send an IP packet to another host on the same LAN, it must place the packet inside a link-layer frame addressed to the destination MAC address. If it does not know that MAC address, it broadcasts an ARP request.

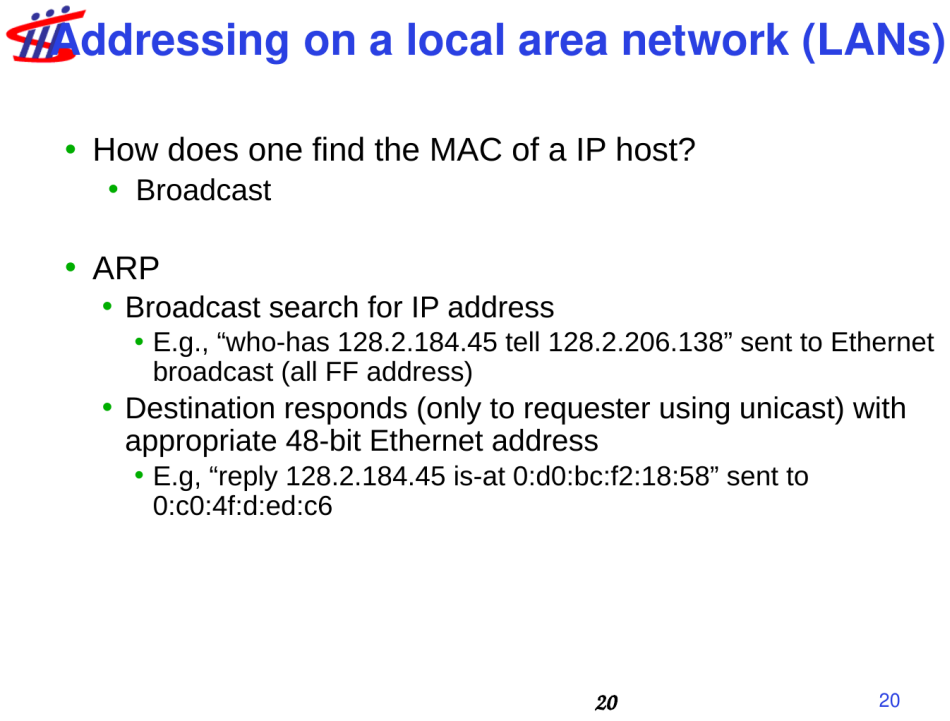


Figure 8: ARP resolves an IP address to a MAC address on a local area network. A host broadcasts a query asking who owns an IP address; the owner replies with its MAC address using unicast.

Algorithm 1: Local Delivery Using ARP

Input: Destination IP address ip_d, IP packet p
Output: A link-layer frame sent on the LAN, or an error

if *ip_d is in the ARP cache* **then**
    $mac_d \leftarrow$ ARP_cache$[ip_d]$;
else
    Broadcast “Who has ip_d?” on the local link;
    Wait for an ARP reply until a timeout;
    if *no reply arrives* **then**
        return error;
    $mac_d \leftarrow$ MAC address in the ARP reply;
    Store (ip_d, mac_d) in the ARP cache;
Build an Ethernet frame with destination mac_d and payload p;
Send the frame on the local link;
return success;

ARP is deliberately local. The entire Internet cannot be one broadcast domain because broadcast does not scale: every query would consume attention and bandwidth from every host. Routers separate broadcast domains. Within a LAN, broadcast is useful; across the wide area, structured IP routing is required.

5.2 Hubs, Collisions, and Shared Media

A hub is conceptually a shared wire. A signal arriving at one port is repeated to all other ports. Hubs do not understand destination addresses; every host hears every frame and discards frames not addressed to itself. This is simple, but inefficient: if one host is transmitting, other hosts sharing the hub cannot transmit without causing a collision.

Early Ethernet handled this with CSMA/CD: Carrier Sense Multiple Access with Collision Detection. A host listens before sending. If it detects a collision, it stops, waits for a random backoff interval, and retries. Under high load, repeated collisions reduce useful throughput.

5.3 Switches and Learning

Switches operate at the data-link layer but are smarter than hubs. A switch learns which MAC address appears behind which physical port by observing the source MAC addresses of frames. Once it knows where a destination MAC address is located, it forwards frames only to the relevant port. This improves both performance and privacy compared with hub-style broadcast.

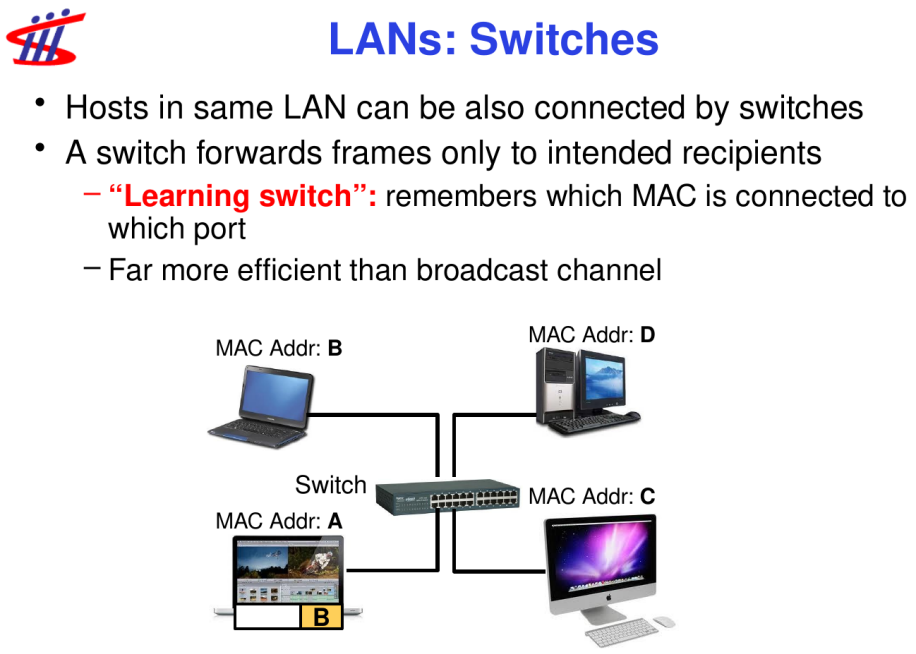


Figure 9: A learning switch forwards frames only to intended recipients once it has learned the mapping from MAC addresses to physical switch ports. This allows multiple pairs of hosts to communicate simultaneously.

A switch's port is a physical connector, not a TCP or UDP port number. This overloaded terminology is common in systems: the same word can mean different things at different layers.

6 Wide Area Networks and Routers

6.1 From LANs to WANs

A wide area network (WAN) connects multiple LANs. Different LANs may use different communication technologies: wired Ethernet, Wi-Fi, cellular, optical links, or specialized data-center fabrics. Routers connect these networks by forwarding packets based on IP destination addresses.

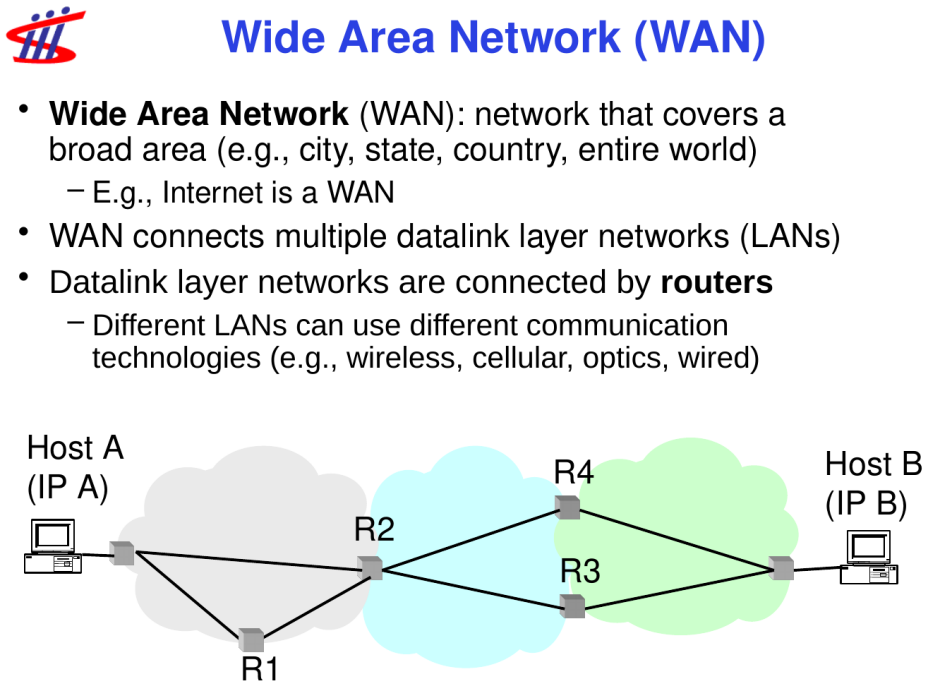


Figure 10: A WAN connects multiple data-link networks. Hosts are attached to local networks, and routers forward packets between those networks using IP-layer information.

Switches and routers differ in their layer and their addressing basis. A switch forwards frames using MAC addresses inside a LAN. A router forwards packets using IP addresses across networks. Routers block ordinary link-layer broadcast; this is what makes distinct IP networks distinct broadcast domains.

6.2 Packet Forwarding and Store-and-Forward

A router receives a packet on an incoming link, reads the IP destination address, consults a forwarding table, and sends the packet to an outgoing link. The forwarding table maps address prefixes to next hops or output interfaces. Longest-prefix matching is used because more specific routes should override broader ones.

In *store-and-forward* switching, a router receives an entire packet before forwarding it. This allows validation and buffering but adds transmission delay at every hop. In *cut-through* forwarding, a device begins forwarding before the whole packet has arrived. Cut-through can reduce latency, but it requires hardware support and may propagate corrupted frames before detecting an error.

6.3 Routing Policy and Business Constraints

It is tempting to imagine routing as a shortest-path problem. At Internet scale, this is incomplete. Paths are determined by topology, link state, routing protocols, business agreements, traffic engineering, security filters, and administrative policies. Autonomous systems announce reachability to IP prefixes, and routes reflect both technical and economic decisions. A path between two universities may pass through education networks, commercial providers, exchange points, and international links; it is not necessarily the geographically shortest path.

This has systems consequences. A failure in an undersea cable, a misconfigured route announcement, a congested inter-provider link, or a policy change can affect application behavior

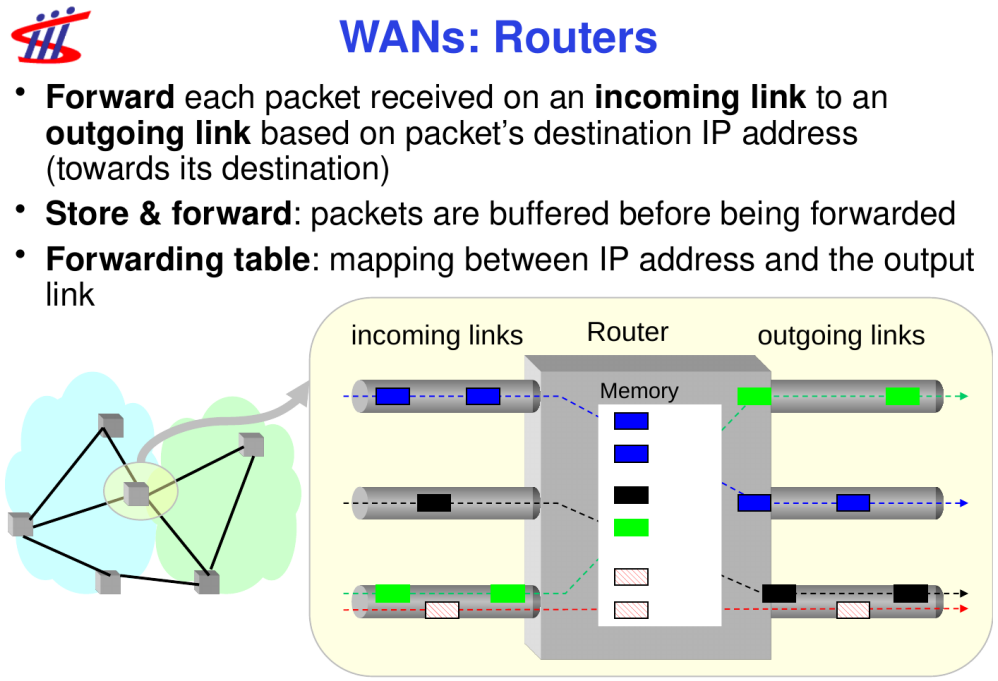


Figure 11: A router forwards packets from incoming links to outgoing links based on the destination IP address. If the outgoing link is busy, packets may be buffered in memory before forwarding.

even when the endpoints are healthy. Content distribution networks (CDNs) and edge caches reduce dependence on distant paths by placing popular content closer to users.

7 DNS: Names as Indirection

7.1 Name Resolution

Humans prefer names; networks route on addresses. DNS, the Domain Name System, maps names such as `iiis.tsinghua.edu.cn` to IP addresses. DNS is not merely a convenience for memory. It is a global distributed database and a powerful layer of indirection.

A typical lookup begins at a client, which asks a local DNS resolver. If the resolver does not have the answer cached, it queries the DNS hierarchy. Root and top-level-domain servers identify where to ask next; authoritative servers for a domain eventually return the relevant record, such as an A record for an IPv4 address or an AAAA record for an IPv6 address.

7.2 Caching, TTL, and Staleness

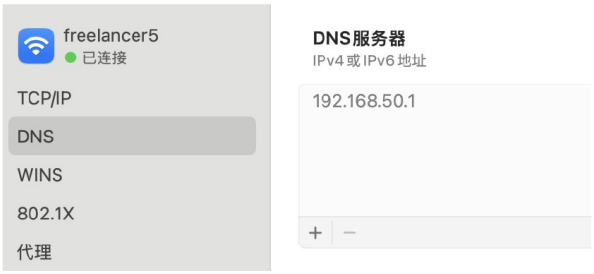
DNS would be too slow if every lookup traversed the hierarchy from the root. Therefore clients, local resolvers, and intermediate resolvers cache records. Caching introduces invalidation problems. DNS records include a time-to-live (TTL), which tells caches how long they may keep an answer. A shorter TTL makes changes propagate faster but increases lookup traffic. A longer TTL reduces load but can cause stale answers after a server moves or fails.

Like many systems abstractions, DNS offers approximate rather than instantaneous consistency. A domain owner can change a record, but old answers may remain in caches until TTL expiration. Some buggy resolvers may cache longer than intended. This is why DNS changes often appear to "take time to propagate."



DNS

- Domain name
- To IP address
- Not just easier to remember
  - A level of indirection
  - E.g. find the closest server
  - E.g. avoid attacks

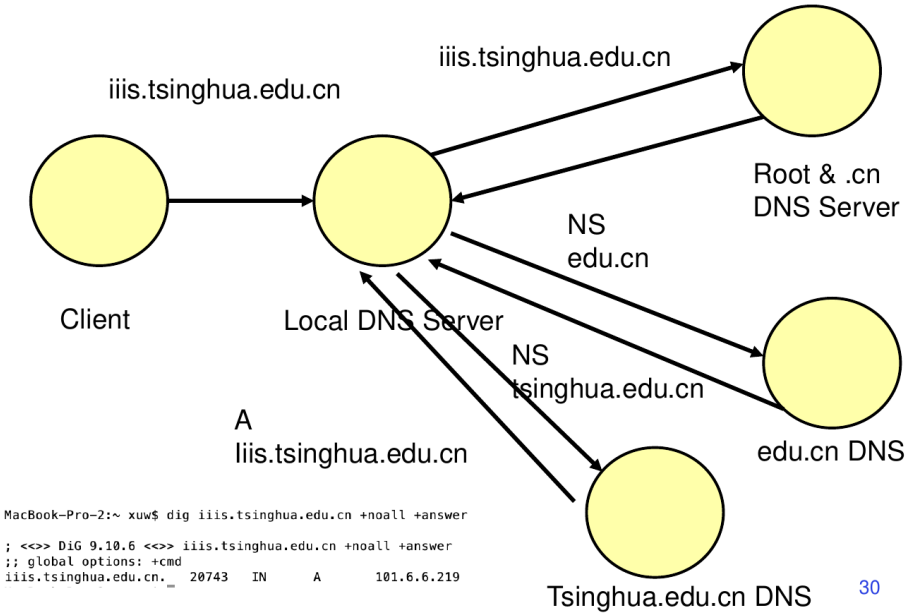


27

Figure 12: DNS maps domain names to IP addresses. This name-to-address mapping is also an indirection layer used for locality, load balancing, fault tolerance, security, and administrative control.



Typical Resolution



30

Figure 13: Hierarchical DNS resolution. A local resolver follows referrals from root or top-level-domain servers to the authoritative server for the requested name, then returns an address record to the client.

7.3 DNS as an Overloaded Mechanism

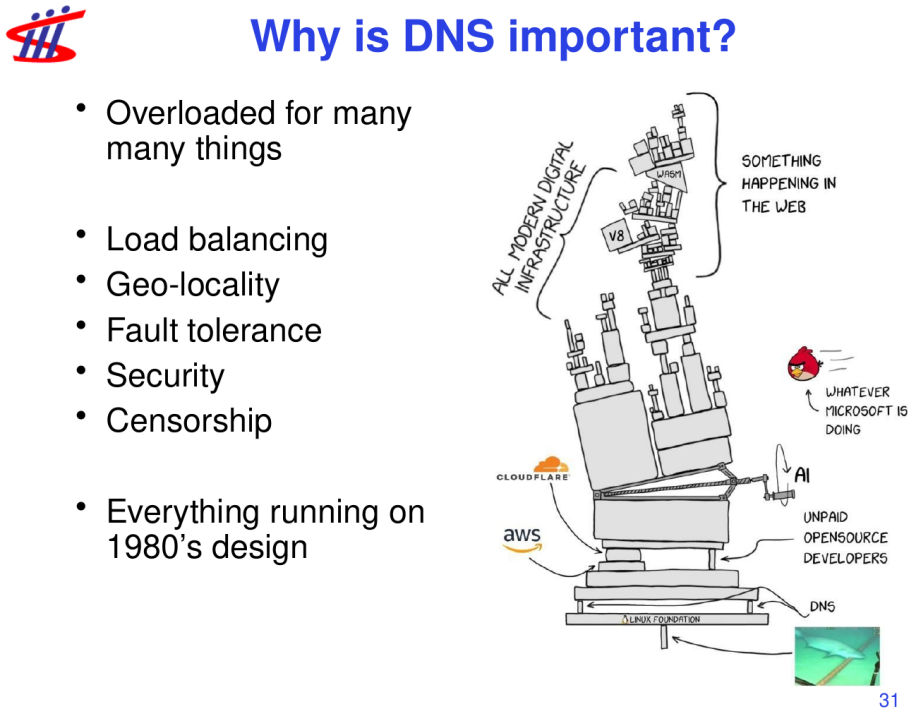


Figure 14: DNS is used for more than naming. It supports load balancing, geo-locality, fault tolerance, security, and sometimes access control, all through an infrastructure originally designed decades ago.

DNS indirection supports useful system designs. A popular service can return different IP addresses to different regions, sending users to nearby data centers. A failed server can be removed from DNS responses. A DDoS-protection provider can hide the origin server behind a protective front-end. However, this power also makes DNS a point of fragility and control: misconfiguration, stale caches, DNS poisoning, or policy-based filtering can prevent users from reaching a service even when the service itself is running.

8 Reliability, Flow Control, and Congestion Control

8.1 Communication Channel Parameters

A communication channel is characterized by more than whether it is connected. Important parameters include latency, capacity, jitter, loss, corruption, and reordering.

Latency is the time for the first bit or a small message to reach the destination. **Capacity**, often called bandwidth, is the maximum number of bits per second the channel can carry. **Jitter** is variation in latency. **Loss** means packets disappear. **Corruption** means packets arrive with damaged bits. **Reordering** means packets arrive in a different order than they were sent.

Applications value these metrics differently. Bulk download cares mainly about capacity. Interactive gaming and remote control care strongly about latency and jitter. AI training may care about both low latency for synchronization and high bandwidth for gradients, parameters, or activation tensors.

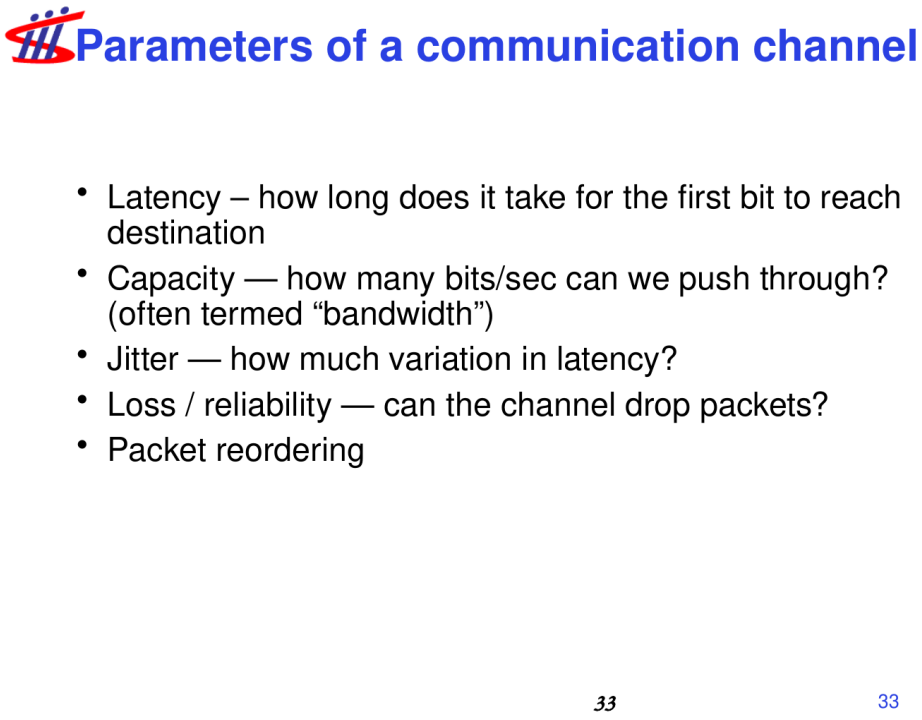


Figure 15: Key parameters of a communication channel: latency, capacity, jitter, loss, and packet reordering. Different applications care about these metrics in different proportions.

8.2 Loss, Corruption, and Reordering

If data may be lost, a sender can use timeouts and retransmission. If no acknowledgment arrives within a reasonable time, the sender retransmits. The fundamental ambiguity is that a timeout does not reveal whether the original packet was lost, delayed, delivered but not acknowledged, or processed by a slow receiver.

If data may be corrupted, the packet can include a checksum or cyclic redundancy check. A receiver that detects corruption usually discards the packet. This converts corruption into loss, which can be handled by retransmission.

If packets may arrive out of order, the protocol can attach sequence numbers. The receiver uses the sequence numbers to reorder data and to detect duplicates caused by retransmission.

8.3 Network Overload

When a router receives packets faster than it can send them, it buffers packets in a queue. Buffering absorbs short bursts and reduces jitter. If the buffer overflows, packets are dropped. Dropping causes retransmission, which adds more traffic, which can cause more dropping. Congestion control tries to avoid or escape this positive feedback loop.

8.4 Stop-and-Wait and the Cost of RTT

The simplest reliable protocol is stop-and-wait. The sender transmits one packet, waits for an acknowledgment, and only then sends the next packet. This is polite but inefficient on long-delay links.

Packet delay includes propagation delay, transmission delay, processing delay, and queuing delay. Propagation delay depends on distance and signal speed. Transmission delay is packet size divided by link capacity. Processing delay is router or host computation. Queuing delay depends on traffic load and buffer occupancy.

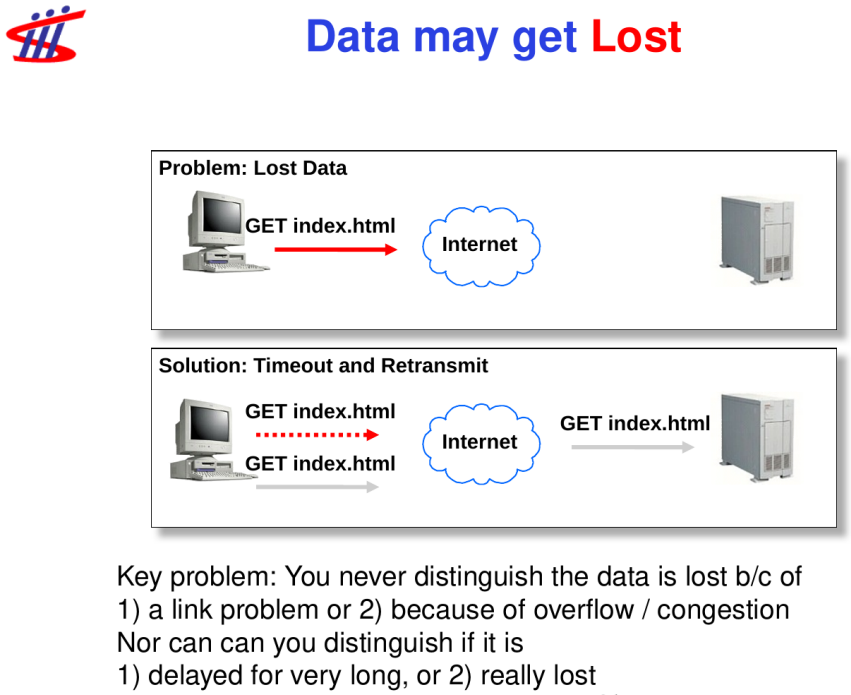


Figure 16: Packet loss is handled by timeout and retransmission, but the sender cannot directly distinguish loss caused by link errors from loss caused by congestion or long delay.

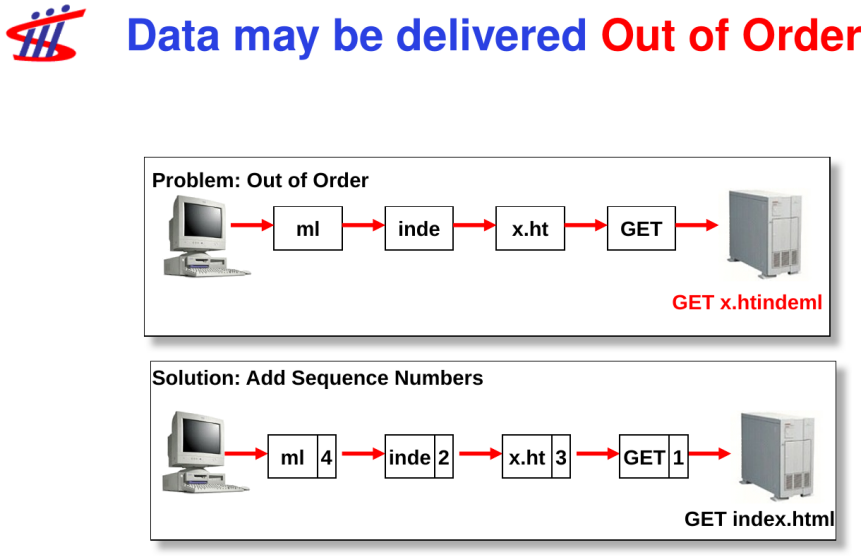


Figure 17: Sequence numbers allow receivers to reconstruct the intended order of data and identify duplicate transmissions.

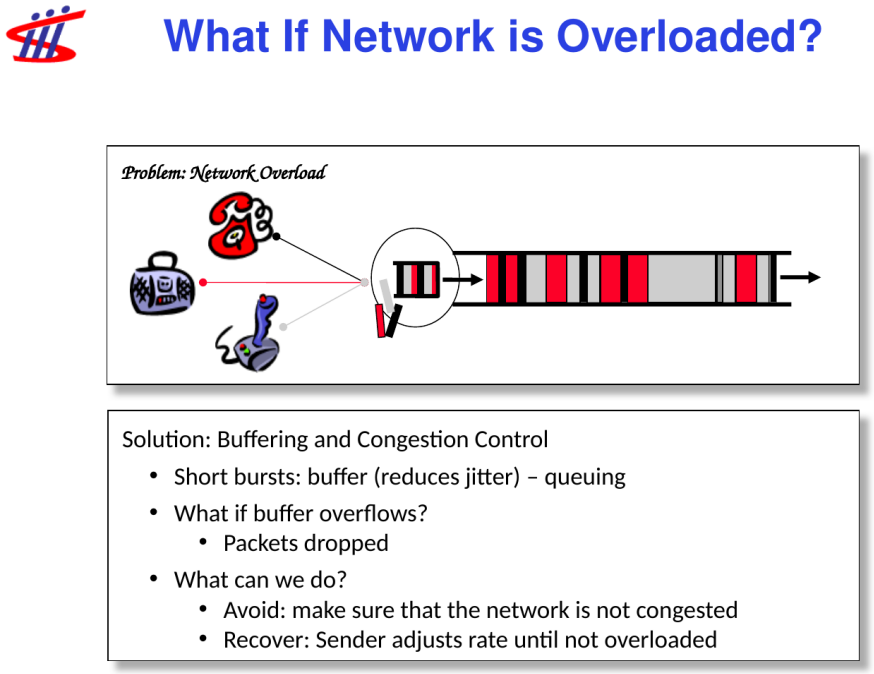


Figure 18: Network overload creates queues. Buffers can absorb short bursts, but if buffers overflow, packets are dropped and retransmission can make congestion worse.

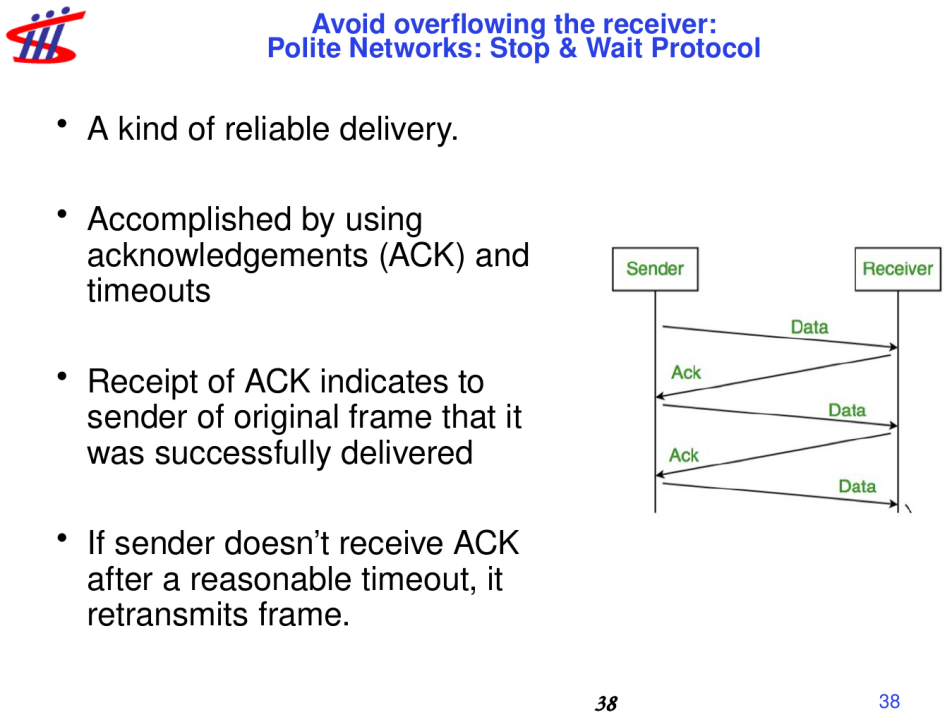


Figure 19: Stop-and-wait uses acknowledgments and timeouts. It avoids overwhelming a receiver but wastes capacity when round-trip time is large.

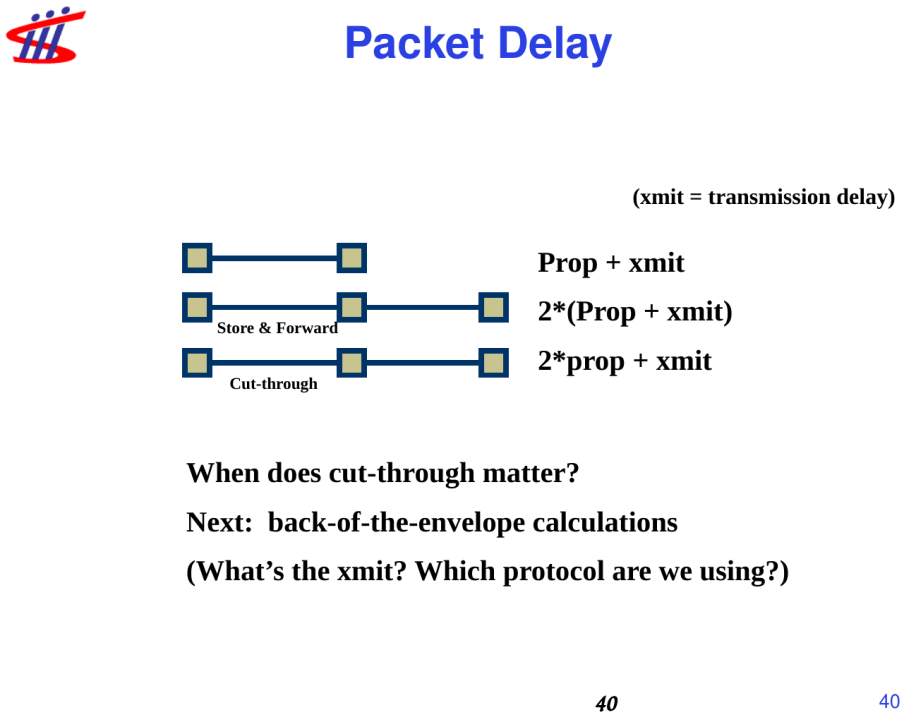


Figure 20: Store-and-forward and cut-through forwarding have different latency behavior. Transmission delay matters when a device waits for a packet to arrive before forwarding it.

A useful back-of-the-envelope calculation shows why stop-and-wait underutilizes fast long-distance links. Suppose a cross-country fiber path is 5000 km. If signal speed in fiber is approximately 2×10^8 m/s, one-way propagation delay is

$$\frac{5 \times 10^6}{2 \times 10^8} = 25 \times 10^{-3} \text{ s} = 25 \text{ ms}.$$

The round-trip time is about 50 ms. If a packet is 1250 bytes, or roughly 10 kbits, and the link speed is 100 Mbps, the transmission delay is

$$\frac{10^4}{10^8} = 10^{-4} \text{ s} = 0.1 \text{ ms}.$$

A stop-and-wait sender transmits 10 kbits and then waits roughly 50.1 ms, for effective throughput

$$\frac{10 \text{ kbits}}{50.1 \text{ ms}} \approx 200 \text{ kbits/s},$$

far below the 100 Mbps link capacity.

8.5 Sliding Windows and TCP Congestion Control

TCP improves on stop-and-wait by allowing a window of packets to be outstanding before acknowledgments arrive. If the window contains W packets, the sender can keep the pipe fuller. To fully utilize a link, the amount of in-flight data should approach the bandwidth-delay product:

$$\text{bandwidth-delay product} = \text{capacity} \times \text{round-trip time}.$$

For a 100 Mbps link with a 50 ms RTT, this product is 5 Mbits, or about 625 KB. A sender with only one 1250-byte packet in flight uses almost none of the available pipe.

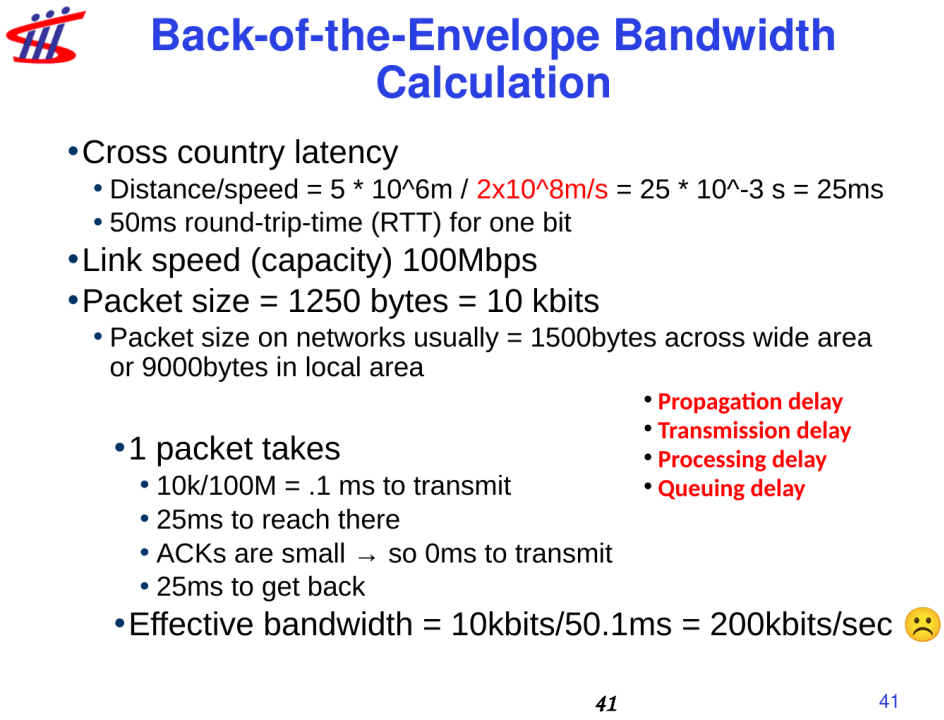


Figure 21: Back-of-the-envelope bandwidth calculation. Even a fast link can be poorly utilized if the sender waits a full round trip after every small packet.

TCP must choose its window without knowing the exact path capacity or competing traffic. It therefore probes. A simplified view is slow start followed by additive-increase/multiplicative-decrease (AIMD). The sender increases the congestion window while acknowledgments arrive. When it detects loss, it treats loss as evidence of congestion and reduces the window sharply.

Algorithm 2: Simplified TCP AIMD Congestion Control

Input: Initial congestion window *cwnd*, slow-start threshold *ssthresh*
Output: An adaptive sending window

```
for each round-trip interval do
    Send up to cwnd bytes or packets permitted by receiver flow control;
    if all expected ACKs arrive then
        if cwnd < ssthresh then
            Increase cwnd rapidly during slow start;
        else
            Increase cwnd additively;
    if loss or timeout is detected then
        ssthresh ← cwnd/2;
        Reduce cwnd multiplicatively;
        Retransmit missing data;
return cwnd;
```

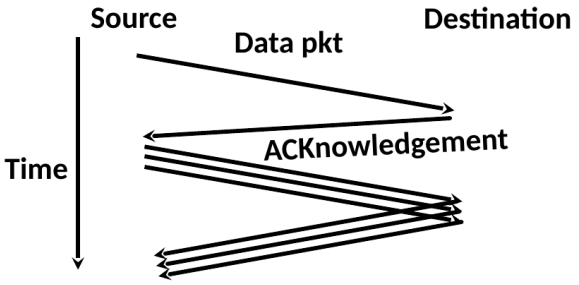
8.6 When TCP Is Not the Right Abstraction

TCP is widely useful, but its assumptions do not fit every network. TCP often treats packet loss as congestion. On a link where random loss is common even without congestion, such as some early mobile data networks or unreliable wireless links, TCP can reduce its window repeatedly



How does it work in reality? (TCP)

This is a *very* incomplete view. :)



What TCP does:
1) Allow W packets before requiring an ACK
2) Need to figure out W
3) Also need to figure out which packets got lost

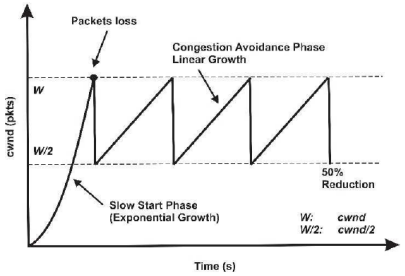
43

Figure 22: A simplified view of TCP: allow a window of packets before requiring acknowledgments, infer lost packets, and adjust the sending rate according to network feedback.



Intuitive TCP flow control

- Retransmit lost packets
  - Keep a sequence number on each packet
- Figuring out the rate
  - Slow start
    - Start with very small sending rate, increase fast
    - Until detect overflow (how?)
  - Slowdown quickly, then increase slowly until overflow again
  - AIMD = additive increment multiplicative decrement
- Congestion control
- Many details + optimizations in networking course



44

Figure 23: Intuitive TCP flow control and congestion control. A sender retransmits lost packets, numbers packets, and adjusts its rate through slow start and AIMD.

and achieve very low throughput. An application using UDP with its own transport logic may perform better if it understands that loss is random rather than congestion-driven.

Low-power sensor networks provide another example. A sensor that sends one small reading every half hour may not want a TCP three-way handshake and reliable stream semantics. The energy cost of connection setup can exceed the value of the measurement. For such applications, sending one datagram and accepting occasional loss may be the better engineering tradeoff.

These examples do not imply that TCP is bad. They show that every abstraction has assumptions. When those assumptions match the workload, the abstraction is powerful; when they do not, the abstraction can get in the way.

9 Layering and the Internet Hourglass

9.1 Why Layering Exists

If every application had to be rewritten for every physical network technology, the Internet would not scale. Layering turns an $n \times m$ compatibility problem into roughly an $n + m$ problem. Applications rely on a common network abstraction, and physical technologies implement that abstraction. This is the same modularity idea used throughout software systems: define an interface, hide implementation details below it, and allow independent innovation above and below.

Definition

A network **layer** provides a service to the layer above, uses the service of the layer below, and communicates with its peer layer on another machine using a protocol.

A *service* is what a layer does. A *service interface* is how the layer above accesses that service. A *protocol* is how peer layers communicate to implement the service. This distinction matters because an implementation can change without changing the service interface, and different implementations can interoperate if they follow the same protocol.

9.2 The Five-Layer Internet Model

The OSI model defines seven layers, but the practical Internet model is often described with five: physical, data link, network, transport, and application. The session and presentation functions of OSI are usually handled inside applications or libraries in the Internet model.

Table 2: The five practical Internet layers.

Layer	Main service	Typical protocols or mechanisms
Physical	Transmit bits over a medium	Copper, fiber, radio, modulation, signal timing
Data link	Transmit frames on one local link or LAN	Ethernet, Wi-Fi, MAC addresses, ARP, CSMA/CD
Network	Deliver packets across multiple networks	IP, routers, forwarding tables, prefix routing
Transport	Deliver data to processes and optionally provide reliability/rate control	TCP, UDP, port numbers, retransmission, flow control
Application	Provide user-level services	HTTP, DNS, SMTP, SSH, RPC, BitTorrent

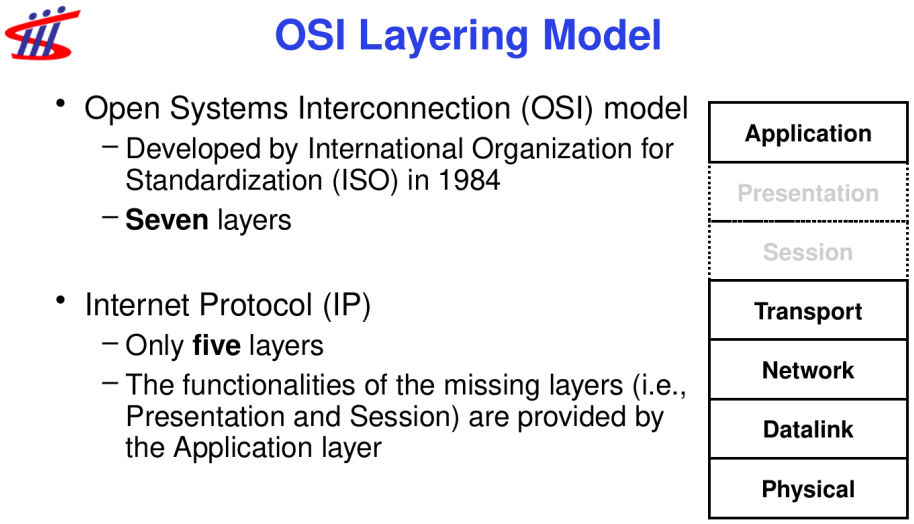


Figure 24: The OSI model has seven layers, while the practical Internet architecture is often taught as five layers. Session and presentation functions are usually folded into the application layer.

Encapsulation is the mechanism by which layers compose. Application data becomes the payload of a transport segment. The transport segment becomes the payload of an IP packet. The IP packet becomes the payload of a data-link frame. At the receiver, headers are removed in the reverse order.

9.3 The Internet Hourglass

The Internet architecture is often called an hourglass. Many applications and transport protocols sit above; many link and physical technologies sit below; IP forms the narrow waist. The narrow waist enables interoperability: any application that can run over IP can use any network technology that carries IP.

This architecture permits simultaneous innovation. A new application can be deployed without changing fiber or Wi-Fi. A new link technology can be deployed without rewriting browsers and web servers. The cost is that changing the waist itself is extremely hard. IPv6 illustrates the difficulty: even when a better addressing architecture is standardized, deployment requires changes across operating systems, routers, applications, network providers, and administrative practices.

9.4 Drawbacks of Layering

Layering hides details, and hiding details can hurt performance. Headers can become large relative to payloads. A tiny application value may be wrapped in application, transport, IP, and link headers, like a small gift inside many layers of packaging. Layers may duplicate functions: a reliable link may carry TCP, which implements reliability again. Different layers may also need the same information, such as timestamps or maximum transmission unit (MTU) sizes.

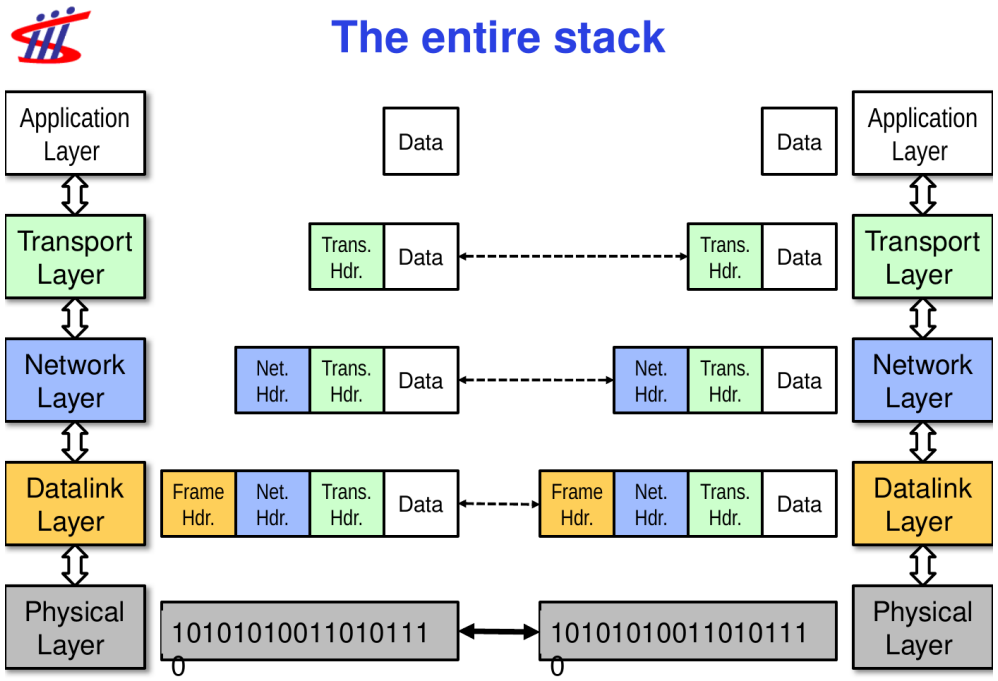


Figure 25: The entire stack encapsulates data by adding headers at each layer. At the destination, the stack removes the headers and delivers the original application data upward.

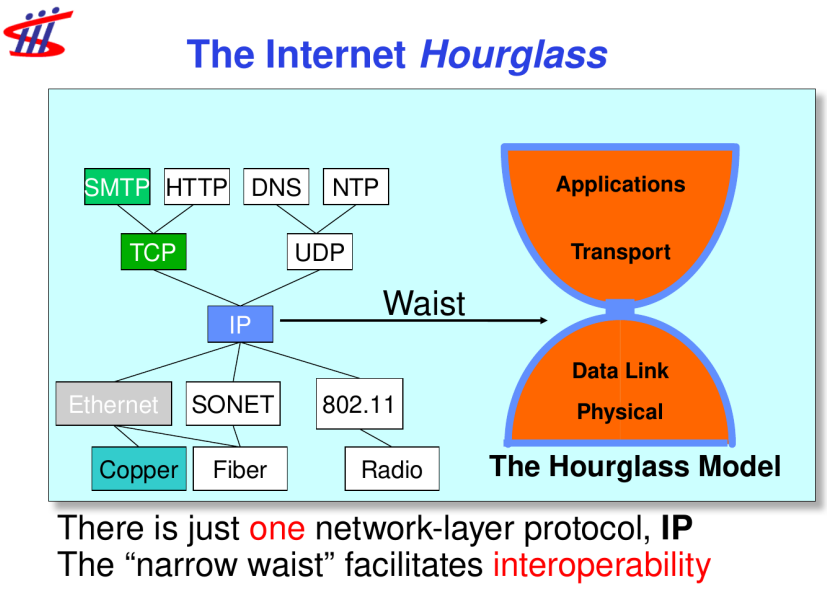


Figure 26: The Internet hourglass. Many applications and transports run above IP, and many physical and data-link technologies run below IP. The narrow waist makes interoperability possible.

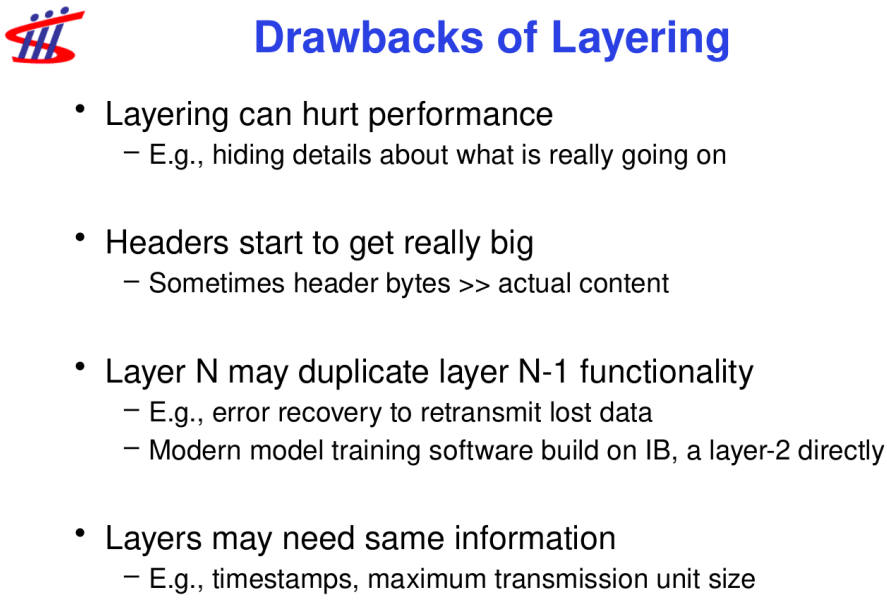


Figure 27: Layering can hurt performance by adding headers, duplicating functionality, and hiding details such as MTU, timestamps, and hardware capabilities.

Deep Dive

The MTU is the maximum transmission unit of a layer. If a frame can carry about 1500 bytes, a packet just below the MTU wastes little capacity. A packet just above the MTU may be fragmented into two frames, wasting much more. Jumbo frames increase the limit, often to around 9000 bytes in local high-performance networks, but the boundary problem remains: exceeding the limit by a small amount can still create an extra frame.

High-performance networks expose the cost of over-abstraction. InfiniBand already provides properties that make reliable low-latency communication possible inside a cluster. Running ordinary TCP/IP over such a fabric may impose header parsing, memory copies, and transport logic that duplicate what the hardware already does. RDMA, remote direct memory access, takes advantage of the fabric by allowing one machine to write directly into a registered memory buffer on another. RDMA can avoid kernel crossings and copies, but it relies on strong assumptions about the underlying network. If packets are lost silently, the remote memory image may be incomplete or the operation may hang.

10 The End-to-End Argument

10.1 Where Should Functionality Live?

The end-to-end argument asks whether a function should be implemented inside the network or at the endpoints. The classic claim is: if an application must implement a correctness function end-to-end anyway, then implementing the same function inside the communication system is redundant unless doing so provides a compelling performance benefit.

Consider reliable delivery. TCP can retransmit lost packets, but an application transferring a file still needs to know whether the complete file was transferred. If the sending process crashes

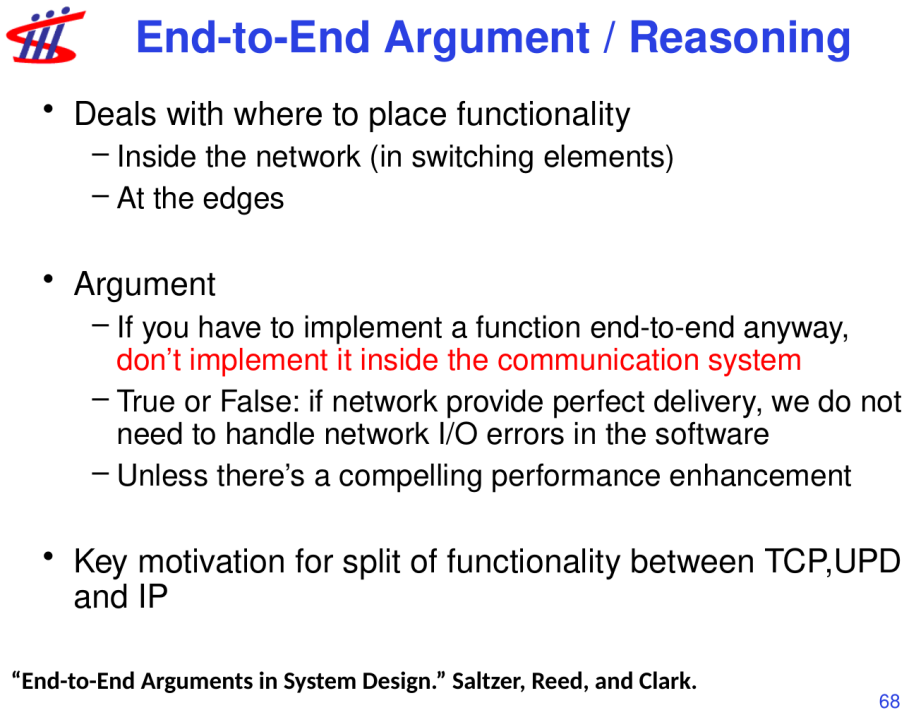


Figure 28: The end-to-end argument concerns where to place functionality. If endpoints must implement a function anyway, lower-layer duplication needs a strong performance justification.

halfway through, TCP cannot transmit data the process never provided. The receiver must still detect that the object is incomplete and handle the failure. Therefore TCP reliability does not eliminate application-level error handling.

Why use TCP reliability at all? Performance. If a 2 MB object loses two packets, TCP can retransmit those packets rather than forcing the application to resend the whole object. Lower-layer recovery sees finer-grained failures earlier and can repair them more cheaply.

Security has a similar structure. Wi-Fi encryption protects the local wireless link, not the entire path. End-to-end encryption such as TLS protects data between application endpoints. Link-layer encryption can still be useful as defense in depth or to protect legacy applications, but it cannot replace application-level protection when the goal is confidentiality from all intermediate networks.

11 Inter-Process Communication and Sockets

11.1 IPC on One Machine and Across the Network

A process is a virtual machine abstraction: it cannot directly see another process's memory. On one machine, the operating system offers IPC mechanisms such as `fork` return values, signals, files, standard input and output, pipes, shared memory segments, and memory-mapped files. Across a network, the basic operating-system interface is the socket.

11.2 The Socket API

A socket is a communication endpoint. In a client-server pattern, the server binds to a port, listens, and accepts incoming connections. The client creates a socket and connects to the server's IP address and port. After a connection is established, both sides can read and write; the data path is bidirectional even though the connection setup roles are asymmetric.

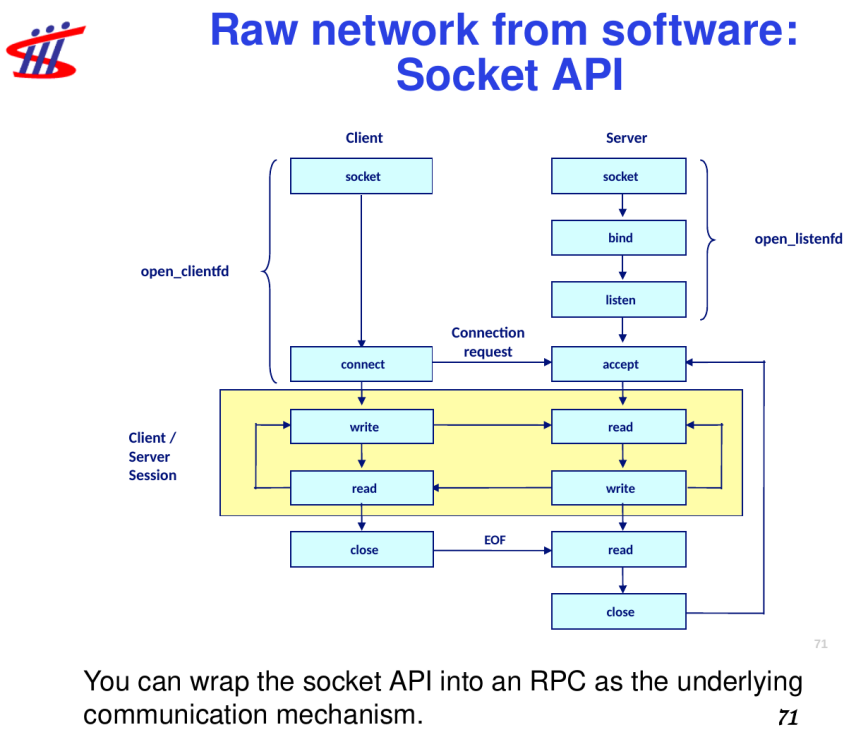


Figure 29: The raw network software interface is the socket API. A server binds, listens, and accepts; a client connects; both sides then read and write bytes over the connection.

```c
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr));
listen(listen_fd, SOMAXCONN);

for (;;) {
    int conn_fd = accept(listen_fd, NULL, NULL);
    if (conn_fd < 0) {
        continue;                    // handle transient errors carefully
    }
    handle_connection(conn_fd);      // read/write bytes
    close(conn_fd);
}
```

Listing 1: A simplified TCP server skeleton using sockets.

```c
int fd = socket(AF_INET, SOCK_STREAM, 0);
connect(fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
write(fd, request_bytes, request_len);
ssize_t n = read(fd, reply_buf, sizeof(reply_buf));
close(fd);
```

Listing 2: A simplified TCP client skeleton using sockets.

Sockets are flexible but low-level. They expose byte streams, not typed values. If a program sends an integer, a string length, a string, and an array, the socket does not preserve those types. The receiver must parse bytes according to an agreed protocol. Manual parsing is fragile: one side may add a field, use a different byte order, misunderstand a length prefix, or fail to validate input. Many security vulnerabilities arise from such parsing mistakes.

11.3 Client-Server and Peer-to-Peer Modes

The words *client* and *server* describe connection roles, not machine size. The server is the side that passively listens and accepts. The client actively connects. Once connected, either side can send. In peer-to-peer systems, the same host may act as both client and server: it listens for requests from others and also connects outward to request data.

12 Remote Procedure Calls

12.1 From Raw Bytes to Typed Calls

Raw sockets resemble programming with untyped byte arrays. High-level languages provide types because programmers need structure: an `int` is not merely four bytes, but four bytes with a representation and meaning. RPC applies this idea to communication. Instead of sending ad hoc bytes, a client calls a procedure with typed arguments, and the RPC system turns the call into request and response messages.

Definition

A **remote procedure call** is a communication abstraction in which a program invokes a procedure on another machine using an interface that resembles a local function call. Under the covers, the RPC runtime sends a request message, executes code on the server, and returns a response message.

12.2 Stubs, Marshalling, and Unmarshalling

The key implementation mechanism is the stub. The client calls a client stub, which marshals typed arguments into a canonical byte representation and sends a request. The server stub receives the request, unmarshals bytes into typed arguments, calls the real server function, marshals the return value, and sends it back. The client stub unmarshals the return value and returns to the caller.

Algorithm 3: RPC Request-Response Flow

Input: Procedure name f, typed arguments $args$
Output: Typed return value or exception
1 Client stub serializes $(f, args)$ into a request message;
2 Client runtime sends the request to the server;
3 Server runtime receives the request;
4 Server stub deserializes $args$ and calls implementation $f(args)$;
5 Server stub serializes the return value or error;
6 Server runtime sends the response to the client;
7 Client stub deserializes the response;
8 **return** typed value or raise exception;

Marshalling must handle byte order, alignment, language differences, object references, strings, arrays, optional fields, and versioning. A simple text representation such as JSON is readable and portable, but it can be verbose and expensive to parse. Binary encodings such as Protocol Buffers are more compact and faster, especially on high-throughput networks where CPU parsing can become the bottleneck.

12.3 IDL and gRPC

An RPC system needs a single source of truth for the interface. An interface definition language (IDL) declares service methods and message types. A compiler then generates matching client

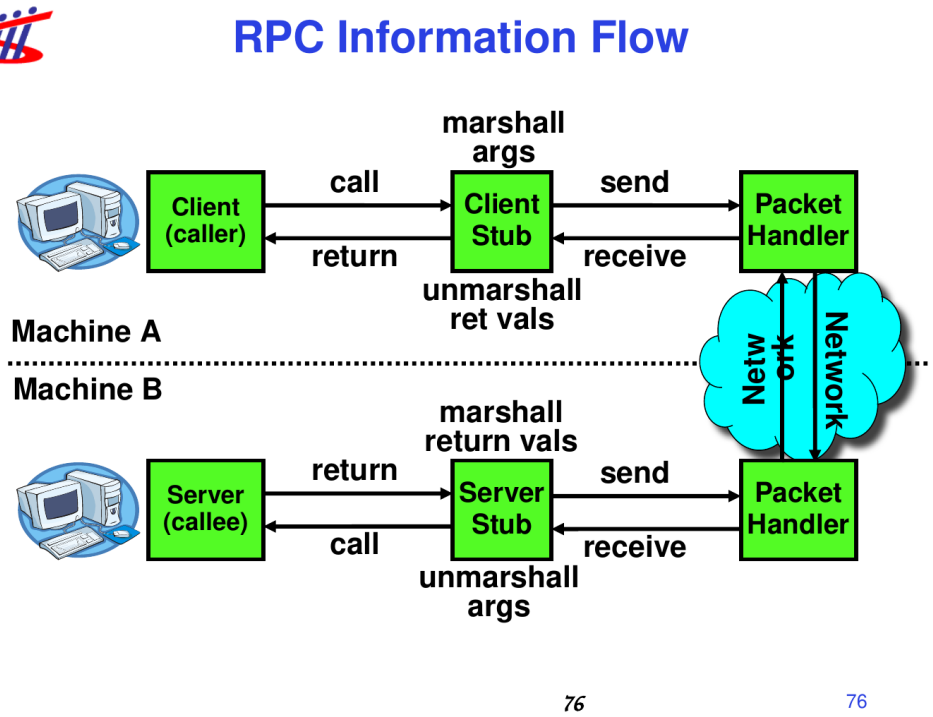


Figure 30: RPC information flow. Client and server stubs marshal and unmarshal typed arguments and return values, while packet handlers and the network carry request and response messages.

and server stubs for one or more programming languages. This avoids the dangerous situation where each side manually guesses the byte layout.

```
syntax = "proto3";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

Listing 3: A small Protocol Buffers interface definition.

The field numbers, such as `1`, are part of the binary encoding and versioning mechanism; they are not ordinary assignments. The protocol compiler generates code for messages, client stubs, server base classes, and serialization logic. The programmer implements the service logic, not the byte-level parsing.



GRPC

- GRPC is a cross-platform, cross-language RPC system developed at Google
- Uses its own IDL (.proto files)

```
// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```

78 78

Figure 31: gRPC uses Protocol Buffers and `.proto` interface definitions to generate cross-language RPC code.



Server and Client Template

```
#include "helloworld.grpc.pb.h"

// Logic and data behind the server's behavior.
class GreeterServiceImpl final : public Greeter::Service {
  Status SayHello(ServerContext* context, const HelloRequest* request,
                  HelloReply* reply) override {
    std::string prefix("Hello ");
    reply->set_message(prefix + request->name());
    return Status::OK;
  }
};
```

IDL compiles to the RPC stub code (.pb.h/.pb.cc code)
As well as a GreeterServiceImpl template

79 79

Figure 32: Generated server and client templates. The IDL compiler emits stub code and service skeletons so the programmer can focus on the actual service implementation.

```cpp
class GreeterServiceImpl final : public Greeter::Service {
public:
    grpc::Status SayHello(grpc::ServerContext* context,
                          const HelloRequest* request,
                          HelloReply* reply) override {
        reply->set_message("Hello, " + request->name());
        return grpc::Status::OK;
    }
};
```

Listing 4: A schematic gRPC service implementation.

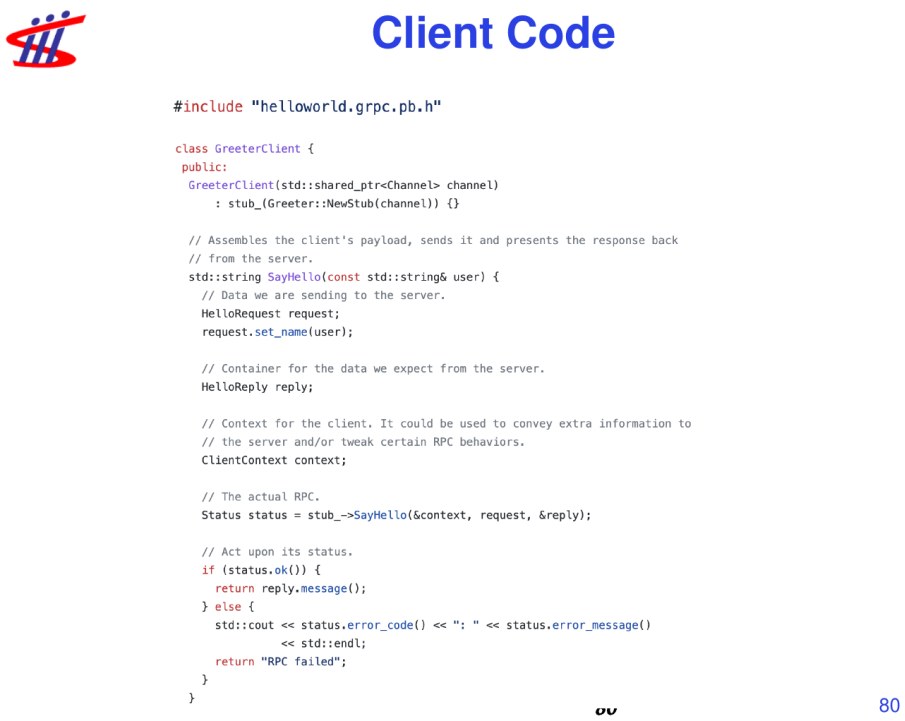


Figure 33: Client-side RPC code calls a generated stub. The code resembles a local call syntactically, but it still performs network communication and can experience network failures.

RPC is a productivity tool, but it should not create the illusion that the network is local memory. A local function call is cheap, synchronous, and shares a fate with the caller. A network RPC is slower, may time out, may execute partially, and may leave the caller and callee in different failure states.

13 RPC Failure Semantics

13.1 RPC Is Not a Local Call

A local procedure call normally fails only if the whole process fails. Caller and callee share the same address space and fate. An RPC crosses a failure boundary. The client may be alive while the server has crashed. The server may complete the operation while the reply is lost. The client may time out without knowing whether the request was delivered. These partial failures are the defining challenge of distributed systems.

Performance also differs. A procedure call is much cheaper than same-machine IPC, which is much cheaper than network RPC. Programmers must therefore design RPC interfaces to be

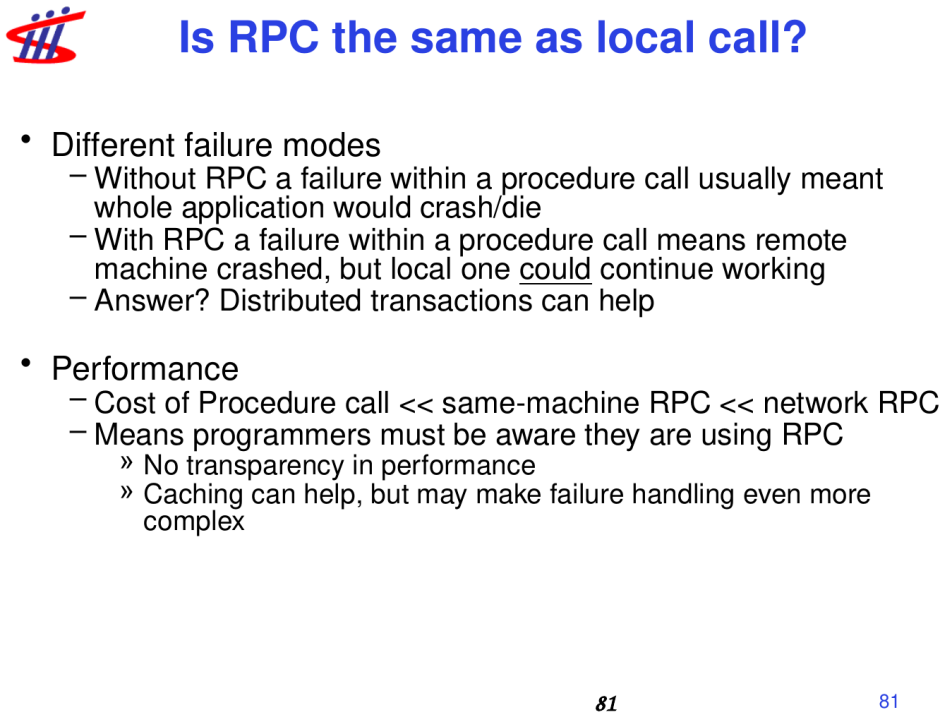


Figure 34: RPC differs from a local call in failure modes and performance. A remote machine may crash independently, and a network RPC costs far more than a local procedure call.

coarse enough to amortize latency. Calling a remote service once per byte or once per small field is usually a serious design error.

13.2 At Least Once, At Most Once, and Exactly Once

When an RPC times out, the system must decide whether to retry. This leads to call semantics.

At least once. The system retries until it obtains evidence that the request executed. This is easy to approximate but may execute the operation multiple times.

At most once. The system ensures that the operation executes no more than once. This avoids duplicates but may report failure even though retry might have succeeded.

Exactly once. The operation appears to execute once and only once. In an asynchronous distributed system with crashes and message loss, this is difficult to provide in a simple, transparent way because the client cannot always distinguish "server did not execute" from "server executed but the reply was lost."

A money-transfer RPC illustrates the danger. If a client asks a bank to transfer 5 yuan and then times out, retrying blindly may transfer 10 yuan if the first request actually succeeded. For non-idempotent operations, the application must participate in the protocol: request identifiers, deduplication tables, transaction logs, or explicit application-level confirmation may be required.



Call Semantics

- Exactly once (same as local IPC)
- At most once
- At least once

- How about if a RPC times out?

82 82

Figure 35: RPC call semantics. A timeout raises the question of whether an operation should be retried, and whether the system can provide at-least-once, at-most-once, or exactly-once behavior.

Algorithm 4: Safe Retry Pattern with Request Identifiers

Input: Client operation op, unique request identifier rid
Output: Result of op or failure

Client sends (rid, op) to the server;
Server checks whether rid is already in its completed-request table;
if *rid is already completed* **then**
    Return the stored result for rid without re-executing op;
else
    Execute op atomically;
    Store $(rid, result)$ durably;
    Return $result$;
if *client times out* **then**
    Client may retry the same (rid, op);
return result or application-level failure;

This pattern is not free. The server must store request identifiers long enough to suppress duplicates, and if the operation changes durable state, the deduplication record must be consistent with that state. This is why RPC semantics connect naturally to transactions and distributed-system recovery.

13.3 Idempotence

Definition

An operation is **idempotent** if executing it once has the same effect as executing it multiple times. Retrying idempotent operations is much safer than retrying non-idempotent operations.

Reading block 1000 from a file is idempotent if the request explicitly includes the offset and length. Advancing an implicit file cursor and reading the next block is not idempotent: a retry may read a later block. Setting a variable to 5 is idempotent; adding 5 is not. Deleting a named file can be made mostly idempotent if the second delete returns an advisory "already gone" error rather than performing a different action.

Good distributed APIs often expose explicit names, offsets, versions, or request identifiers so that operations become easier to retry. This is a systems design lesson: interfaces determine failure semantics.

14 Summary

Networking is a layered system for sharing communication resources among many applications, hosts, and organizations. Links transmit bits; data-link protocols transmit frames and resolve local destinations; IP routes packets across networks; transport protocols identify processes and optionally provide reliability and congestion control; application protocols define the meaning of messages.

Multiplexing makes shared links efficient. Packet switching statistically multiplexes bursty traffic, while circuit switching reserves resources. Addressing works at multiple scopes: MAC addresses identify local interfaces, IP addresses support routing across networks, and port numbers identify processes. ARP uses local broadcast to map IP addresses to MAC addresses; switches learn MAC-to-port mappings; routers forward packets by IP prefixes; DNS maps human-readable names to addresses and provides a powerful layer of indirection.

Reliability requires explicit mechanisms: checksums for corruption, sequence numbers for ordering and duplicates, acknowledgments and timeouts for loss, and congestion-control algorithms for overload. Stop-and-wait is simple but wastes bandwidth on high-RTT paths; window-based protocols such as TCP keep more data in flight and adjust rates through feedback. TCP is useful but not universal; some networks and applications need different transport semantics.

Layering enables the Internet hourglass: many applications above, many technologies below, and IP as a narrow waist. The same abstraction that enables interoperability can hide performance details, duplicate functionality, and make changes difficult. The end-to-end argument provides a design principle for placing functionality: implement correctness at endpoints when endpoints must check it anyway, and duplicate functionality inside the network only for strong performance reasons.

At the software interface, sockets expose raw byte streams and require explicit protocols. RPC raises the abstraction by making communication resemble typed procedure calls with generated stubs and marshalling. However, RPC is not local execution. Network latency, independent crashes, timeouts, retries, partial failures, and ambiguous execution semantics force programmers to reason about idempotence, at-least-once behavior, at-most-once behavior, and the difficulty of exactly-once semantics.

15 Quizzes & Exercises

15.1 Short-Answer Quiz

1. Why does packet switching usually achieve better utilization than circuit switching for bursty data traffic?
2. Explain the difference between flow control and congestion control. Give one example of each problem.
3. Why are MAC addresses not sufficient for Internet-scale routing?
4. What information is needed to identify a TCP or UDP endpoint globally?
5. What problem does ARP solve, and why is it based on broadcast?
6. How does a learning switch discover which MAC address is behind which physical port?
7. Why do routers separate broadcast domains?
8. What is the role of TTL in DNS caching? What tradeoff does a DNS operator face when choosing a TTL?
9. Give two examples of how DNS is used as an indirection layer beyond merely remembering names.
10. Define propagation delay, transmission delay, processing delay, and queuing delay.
11. Why can stop-and-wait achieve very poor throughput on a path with large RTT even if the link capacity is high?
12. What does the bandwidth-delay product measure, and why is it relevant to TCP windows?
13. Why does TCP reduce its sending rate after packet loss? Give a scenario where this assumption can be wrong.
14. What are the five practical layers of the Internet architecture, and what service does each provide?
15. What is the Internet hourglass model? What is its main advantage and its main disadvantage?
16. State the end-to-end argument in your own words.
17. Why is raw socket programming error-prone compared with RPC or a structured protocol library?
18. What are marshalling and unmarshalling in RPC?
19. Why can a timeout after an RPC not tell the client whether the server executed the operation?
20. Compare at-least-once, at-most-once, and exactly-once RPC semantics.

15.2 Calculation Exercises

1. A path has one-way propagation delay of 30 ms, link capacity of 1 Gbps, and packet size of 1500 bytes. If a stop-and-wait protocol sends one packet per RTT and ACK transmission time is negligible, what is the effective throughput?

2. A TCP connection has RTT 80 ms and bottleneck capacity 200 Mbps. Estimate the bandwidth-delay product in bits and bytes. About how many 1500-byte packets must be in flight to fill the pipe?

3. Suppose an application sends 20-byte messages over a stack that adds 14 bytes of Ethernet header, 20 bytes of IP header, 20 bytes of TCP header, and 30 bytes of application framing. What fraction of each transmitted frame is useful application payload, ignoring lower-level preambles and checksums?

4. A DNS record has TTL 300 seconds. At time 0, the authoritative server changes the record. What is the latest time a standards-compliant cache that fetched the old value just before time 0 may keep returning the old value?

15.3 Design Questions

1. Design an RPC method for transferring money. What request fields would you include to avoid duplicate transfers after retries? Which state must be stored durably on the server?

2. A sensor wakes every 30 minutes and sends a 16-byte temperature reading. Would you use TCP or UDP? Explain the tradeoff among energy, reliability, and complexity.

3. A data-center application runs on an InfiniBand network and sends large tensors between GPUs. Explain why ordinary TCP/IP may waste performance and when RDMA is appropriate.

4. A collaborative editor stores documents in a file system. Why are ordinary read/write file semantics insufficient for multi-user editing? What application-level protocol features might be needed?

5. You are building a service behind DNS load balancing. A server fails. What happens if DNS TTL is one day? What happens if TTL is one second? Discuss both availability and resolver load.

15.4 Programming Exercises

1. Write a small `.proto` file for a key-value service with `Get`, `Put`, and `Delete` methods. Decide which methods are idempotent and explain why.

2. Implement a simple message framing format over a byte stream: a 4-byte network-order length followed by payload bytes. Explain how the receiver should handle partial reads, invalid lengths, and endianness.

3. Modify the safe retry algorithm in this note so that completed request identifiers expire after a time limit. What correctness risk does expiration introduce?